

UNIVERSITY OF OKARA

Department of Computer Science

Course: Artificial Intelligence • BS Computer Science • Practical Implementation Task — Worked Example

IMPLEMENTATION TASK 1

Formulate a Problem as a Search Problem and Implement BFS, DFS and UCS

Artificial Intelligence — Practical Implementation Task | Worked Example: Maze / Grid Search

Instructor	Dr. Ghulam Ali	Task Number	Implementation Task 1
Designation	Associate Professor of Computer Science	Total Marks	20
Course	Artificial Intelligence	Issue / Due Date	

STUDENT INFORMATION			
Student Name		Roll No.	
Semester		Section	
Date		Marks	/ 20

About this Worked Example

This copy of the work file has been **completed for a sample problem (Option D — Maze / Grid Search)** to show students the expected level of detail.

All sample answers appear in **blue text**. Every trace, path, cost and screenshot below was produced by actually running the complete program given in **Appendix A** ([maze_search.py](#)).

Students must choose and formulate **their own problem** — copying this maze or its code for submission is not acceptable.

SECTION 1 | TASK OVERVIEW

In this task you will take a real-world problem and **formulate it as a classical AI search problem**. You will then implement three uninformed search strategies and use them to solve your problem:

1. **Breadth-First Search (BFS)**
2. **Depth-First Search (DFS)**
3. **Uniform-Cost Search (UCS)**

The purpose of this task is two-fold:

- **Problem formulation** — describing a problem precisely in terms of states, actions, transition model, goal test and path cost.
- **Search algorithm implementation and tracing** — learning how each algorithm manages its frontier, why the algorithms return different solutions, and when each one is optimal.

Students must write their own implementation of each algorithm rather than calling a ready-made graph-search function.

SECTION 2 | LEARNING OBJECTIVES

1. Formulate a real-world problem as a search problem.
2. Identify states and actions.
3. Define an initial state.
4. Define a goal state / goal test.
5. Define a transition model.
6. Define path costs.
7. Implement BFS.
8. Implement DFS.
9. Implement UCS.
10. Trace the behaviour of each algorithm.
11. Compare their resulting paths and costs.
12. Explain the difference between BFS, DFS and UCS.

SECTION 3 | PROBLEM CHOICE

Option	Problem Domain	Example Idea	Select
A	University / Campus Navigation	Move between buildings of the University of Okara campus.	☐
B	Robot Navigation	A robot moves between rooms or cells of a warehouse floor.	☐
C	City Route Finding	Find a route between towns/cities with road distances as edge costs.	☐
D	Maze / Grid Search	An agent moves through a grid maze with walls; cells have different costs.	☒
E	Puzzle Search Problem	Water-jug, missionaries & cannibals, small sliding puzzle.	☐
F	Other (instructor-approved)	Proposed problem approved by the instructor.	☐

Selected Option (A–F)	D — Maze / Grid Search (weighted 4 × 4 maze with water and mud cells)
Instructor Approval (Option F only)	Not required (Option D).

SECTION 4 | PROBLEM FORMULATION

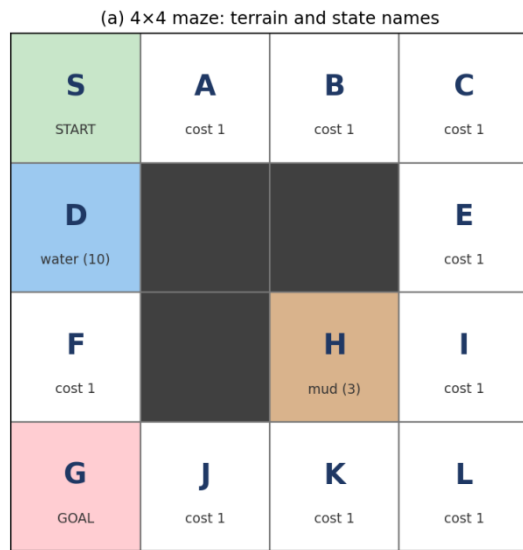
Problem Name	Weighted Maze Navigation (4 × 4 grid)
Problem Description	An agent is placed at the START cell (top-left) of a 4 × 4 maze and must reach the GOAL cell (bottom-left). Some cells are walls and cannot be entered. The remaining cells have different terrain: normal floor, mud (slow) and water (very slow). The direct route down the left side passes through water, while the long route around the maze uses only dry floor. The task is to find a path to the goal and compare BFS, DFS and UCS on it.
Initial State	The agent is in cell S = (row 0, column 0).
Goal State	The agent is in cell G = (row 3, column 0).
State Representation	A state is the agent's position (row, col). For readability each open cell is given a name: S, A, B, C, D, E, F, H, I, J, K, L, G (letter G is reserved for the goal). Maze layout (row 0 at top): S . . . / W # # . / . # M . / G . . . — 13 reachable states.
Actions / Operators	Up, Left, Right, Down (generated in this order). An action is applicable only if the target cell is inside the grid and is not a wall (#).
Transition Model	RESULT((r, c), Up) = (r-1, c); Left = (r, c-1); Right = (r, c+1); Down = (r+1, c). Example: RESULT(S, Right) = A; RESULT(S, Down) = D; RESULT(I, Left) = H.
Path Cost	Step cost = terrain cost of the cell being entered: floor (., S, G) = 1, mud M = 3, water W = 10. Path cost g(n) = sum of step costs along the path. Example: S→D→F→G = 10 + 1 + 1 = 12.
Goal Test	state == G, i.e. position == (3, 0). The test is applied when a node is selected for expansion .

SECTION 5 | STATE-SPACE / GRAPH

Requirements — check

- 13 states (minimum 8) ✓
- One start state S and one goal state G ✓
- Multiple paths from S to G (left route through water, right route with or without mud) ✓
- Every edge has a cost ✓
- Fewer-edge path is more expensive than the long path ✓ (needed for Section 10)

Student State-Space / Graph



(b) State-space graph with step costs
(cost = terrain cost of the cell entered)

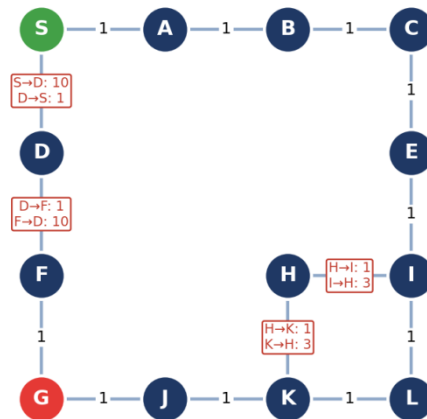


Figure 1. (a) The maze with terrain; (b) the state-space graph. Red labels show connections whose cost depends on direction (entering water or mud is expensive).

Graph Summary (Adjacency List with Costs)

Node	Neighbours (with edge cost)	Start / Goal?
S	A(1) [Right], D(10) [Down]	Start
A	S(1) [Left], B(1) [Right]	—
B	A(1) [Left], C(1) [Right]	—
C	B(1) [Left], E(1) [Down]	—
D	S(1) [Up], F(1) [Down]	—
E	C(1) [Up], I(1) [Down]	—
F	D(10) [Up], G(1) [Down]	—
H	I(1) [Right], K(1) [Down]	—
I	E(1) [Up], H(3) [Left], L(1) [Down]	—
G	F(1) [Up], J(1) [Right]	Goal
J	G(1) [Left], K(1) [Right]	—
K	H(3) [Up], J(1) [Left], L(1) [Right]	—
L	I(1) [Up], K(1) [Left]	—

Total number of nodes: **13** Total number of connections: **14** (26 directed edges) Directed / Undirected: **Directed** (moves are reversible but the cost of entering W or M differs by direction)

SECTION 6 | BFS IMPLEMENTATION

Required Behaviour

- Start from the initial state.
- Maintain a **frontier** of nodes waiting to be expanded.

- Use **FIFO (queue)** behaviour — `collections.deque, popleft()`.
- Track **explored / visited** states (`parent` dictionary = reached set).
- Stop when the goal is reached.
- **Reconstruct and display** the final path using parent pointers.

Sample Implementation — `bfs(graph, start, goal)`

```

1 def bfs(graph, start, goal):
2     frontier = deque([start])          # FIFO queue
3     parent = {start: None}             # also acts as the "reached" set
4     explored = []                      # expansion order
5     trace = []
6
7     while frontier:
8         node = frontier.popleft()       # oldest node first
9         explored.append(node)
10        current_path = reconstruct(parent, node)
11
12        if node == goal:                 # goal test when the node is expanded
13            trace.append((node, list(frontier), list(explored),
14                          current_path, path_cost(graph, current_path)))
15            return current_path, len(explored), trace
16
17        for child, _, _ in graph[node]:
18            if child not in parent:      # not explored and not in frontier
19                parent[child] = node
20                frontier.append(child)
21
22        trace.append((node, list(frontier), list(explored),
23                      current_path, path_cost(graph, current_path)))
24    return None, len(explored), trace

```

Output (from program execution)

BFS Final Path	S → D → F → G
Path Length (edges)	3
Path Cost	12
Nodes Expanded	7
Order of Expansion	S, A, D, B, F, C, G

SECTION 7 | DFS IMPLEMENTATION

Required Behaviour

- Start from the initial state.
- Use **stack / LIFO** behaviour — a Python list with `append()` / `pop()`.
- Track **explored / visited** states in a set.
- **Avoid infinite loops** — a node already in the explored set is never expanded again.
- Stop when the goal is reached.
- Reconstruct the final path (each stack entry carries its own path).

Neighbour order: children are pushed in **reverse** action order, so the first action (**Up**, then Left, Right, Down) is on top of the stack and is explored first.

Sample Implementation — `dfs(graph, start, goal)`

```

1 def dfs(graph, start, goal):
2     frontier = [(start, [start])]      # stack of (node, path to node)
3     explored = []
4     explored_set = set()                # cycle detection
5     trace = []

```

```

6
7     while frontier:
8         node, path = frontier.pop()    # newest node first
9         if node in explored_set:      # already expanded via another path
10            continue
11        explored.append(node)
12        explored_set.add(node)
13
14        if node == goal:
15            trace.append((node, [n for n, _ in frontier], list(explored),
16                          path, path_cost(graph, path)))
17            return path, len(explored), trace
18
19        # Push children in REVERSE action order so the first action
20        # (Up, then Left, Right, Down) ends up on top of the stack.
21        for child, _ in reversed(graph[node]):
22            if child not in explored_set:
23                frontier.append((child, path + [child]))
24
25        trace.append((node, [n for n, _ in frontier], list(explored),
26                      path, path_cost(graph, path)))
27    return None, len(explored), trace

```

Output (from program execution)

DFS Final Path	S → A → B → C → E → I → H → K → J → G
Path Length (edges)	9
Path Cost	11
Nodes Expanded	10
Order of Expansion	S, A, B, C, E, I, H, K, J, G

SECTION 8 | UCS IMPLEMENTATION

Required Behaviour

- Maintain the **cumulative path cost** $g(n)$ for each frontier node.
- Always **expand the lowest-cost** frontier node.
- Use a **priority queue** — Python's **heapq**.
- Track the **best-known path cost** (**best_cost**) and skip stale queue entries.
- Reconstruct the final path.
- Display the **total path cost**.

Sample Implementation — ucs(graph, start, goal)

```

1 def ucs(graph, start, goal):
2     counter = 0                                # tie-breaker: FIFO among equal costs
3     frontier = [(0, counter, start)]           # (g, order, node)
4     best_cost = {start: 0}                     # cheapest known cost to each node
5     parent = {start: None}
6     explored = []
7     explored_set = set()
8     trace = []
9
10    def frontier_view():
11        """Live frontier entries (stale duplicates hidden), cheapest first."""
12        live = [(g, k, n) for (g, k, n) in frontier
13                if n not in explored_set and g == best_cost[n]]
14        return [f"{n}({g})" for g, k, n in sorted(live)]
15
16    while frontier:
17        g, _, node = heapq.heappop(frontier)    # lowest path cost first
18        if node in explored_set or g > best_cost[node]:
19            continue                            # stale entry - skip
20        explored.append(node)
21        explored_set.add(node)
22        current_path = reconstruct(parent, node)

```

```

23
24     if node == goal:
25         trace.append((node, frontier_view(), list(explored), current_path, g))
26         return current_path, g, len(explored), trace
27
28     for child, step, _ in graph[node]:
29         new_cost = g + step
30         if child not in explored_set and new_cost < best_cost.get(child, float("inf")):
31             best_cost[child] = new_cost      # found a cheaper way
32             parent[child] = node
33             counter += 1
34             heapq.heappush(frontier, (new_cost, counter, child))
35
36     trace.append((node, frontier_view(), list(explored), current_path, g))
37     return None, float("inf"), len(explored), trace

```

Output (from program execution)

UCS Final Path	S → A → B → C → E → I → L → K → J → G
Path Length (edges)	9
Total Path Cost	9
Nodes Expanded	11
Order of Expansion	S, A, B, C, E, I, L, K, H, J, G

SECTION 9 | SEARCH TRACE

The tables below were copied from the program output (Appendix A, function `show_trace`). **Frontier** and **Explored** show the state **after** the current node has been expanded.

9.1 BFS Trace

Frontier is a FIFO queue: the **left-most** node is removed next.

Step	Current Node	Frontier	Explored	Current Path	Cost
1	S	A, D	S	S	0
2	A	D, B	S, A	S → A	1
3	D	B, F	S, A, D	S → D	10
4	B	F, C	S, A, D, B	S → A → B	2
5	F	C, G	S, A, D, B, F	S → D → F	11
6	C	G, E	S, A, D, B, F, C	S → A → B → C	3
7	G	E	S, A, D, B, F, C, G	S → D → F → G	12

Observation on BFS behaviour

Nodes leave the queue in the order they were added, so BFS expands level by level: S (depth 0), then A and D (depth 1), then B and F (depth 2), then C and G (depth 3). It reaches G at depth 3 through the water cell and ignores the cost of 12. The cheap route (depth 9) is never reached.

9.2 DFS Trace

Frontier is a LIFO stack: the **right-most** node (top of stack) is removed next.

Step	Current Node	Frontier	Explored	Current Path	Cost
1	S	D, A	S	S	0
2	A	D, B	S, A	$S \rightarrow A$	1
3	B	D, C	S, A, B	$S \rightarrow A \rightarrow B$	2
4	C	D, E	S, A, B, C	$S \rightarrow A \rightarrow B \rightarrow C$	3
5	E	D, I	S, A, B, C, E	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow E$	4
6	I	D, L, H	S, A, B, C, E, I	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow I$	5
7	H	D, L, K	S, A, B, C, E, I, H	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow I \rightarrow H$	8
8	K	D, L, L, J	S, A, B, C, E, I, H, K	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow I \rightarrow H \rightarrow K$	9
9	J	D, L, L, G	S, A, B, C, E, I, H, K, J	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow I \rightarrow H \rightarrow K \rightarrow J$	10
10	G	D, L, L	S, A, B, C, E, I, H, K, J, G	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow I \rightarrow H \rightarrow K \rightarrow J \rightarrow G$	11

Observation on DFS behaviour

DFS keeps following the newest child: it goes right along the top row, down the right column and then turns into the mud cell H (Left is tried before Down). D stays at the bottom of the stack the whole time. L appears twice in the frontier because it was pushed from I and from K; the explored-set check prevents it from being expanded twice. DFS returns a 9-edge path of cost 11, which is neither shortest nor cheapest.

9.3 UCS Trace

Frontier is a priority queue ordered by $g(n)$, shown as node(cost), cheapest first.

Step	Current Node	Frontier	Explored	Current Path	Cost
1	S	A(1), D(10)	S	S	0
2	A	B(2), D(10)	S, A	$S \rightarrow A$	1
3	B	C(3), D(10)	S, A, B	$S \rightarrow A \rightarrow B$	2
4	C	E(4), D(10)	S, A, B, C	$S \rightarrow A \rightarrow B \rightarrow C$	3
5	E	I(5), D(10)	S, A, B, C, E	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow E$	4
6	I	L(6), H(8), D(10)	S, A, B, C, E, I	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow I$	5
7	L	K(7), H(8), D(10)	S, A, B, C, E, I, L	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow I \rightarrow L$	6
8	K	H(8), J(8), D(10)	S, A, B, C, E, I, L, K	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow I \rightarrow L \rightarrow K$	7
9	H	J(8), D(10)	S, A, B, C, E, I, L, K, H	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow I \rightarrow H$	8
10	J	G(9), D(10)	S, A, B, C, E, I, L, K, H, J	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow I \rightarrow L \rightarrow K \rightarrow J$	8
11	G	D(10)	S, A, B, C, E, I, L, K, H, J, G	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow I \rightarrow L \rightarrow K \rightarrow J \rightarrow G$	9

Observation on UCS behaviour

D(10) enters the frontier at step 1 but always stays last, because every dry cell on the long route is cheaper. At step 6, H is reached for cost 8 through the mud, while L (6) and K (7) are cheaper and are expanded first. At step 10 G(9) enters the frontier; it is expanded (and the goal test succeeds) at step 11 before D(10), so UCS returns the optimal cost 9.

SECTION 10 | IMPORTANT EXPERIMENT**Design Requirement**

Design a weighted graph in which “the path containing fewer edges is NOT necessarily the least-cost path.”

Experiment Graph

The same maze as Section 5 (Figure 1) satisfies the requirement:

- **Fewer-edge path:** $S \rightarrow D \rightarrow F \rightarrow G = 3$ edges, cost $10 + 1 + 1 = 12$ (passes through water).
- **Least-cost path:** $S \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow I \rightarrow L \rightarrow K \rightarrow J \rightarrow G = 9$ edges, cost **9** (dry floor only).
- **Mud trap:** $S \rightarrow \dots \rightarrow I \rightarrow H \rightarrow K \rightarrow J \rightarrow G = 9$ edges, cost **11** (passes through mud).

Comparison Table

Algorithm	Returned Path	Path Length	Total Cost	Nodes Expanded	Optimal?
BFS	$S \rightarrow D \rightarrow F \rightarrow G$	3	12	7	No
DFS	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow I \rightarrow H \rightarrow K \rightarrow J \rightarrow G$	9	11	10	No
UCS	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow I \rightarrow L \rightarrow K \rightarrow J \rightarrow G$	9	9	11	Yes

Discussion of Results

BFS found the path with the fewest edges (3) and expanded the fewest nodes (7), but it is not optimal because it treats every step as equal and walks straight through the water (cost 12).

DFS found a long path (9 edges) of cost 11. Its answer depends only on the order of actions: because Left is tried before Down at I, it walked into the mud cell. It is neither shortest nor optimal.

UCS expanded the most nodes (11) because it explored every cheap cell before paying 10 to enter the water, but it returned the least-cost path (cost 9). This shows that the shortest path in edges (BFS) and the least-cost path (UCS) can be different.

SECTION 11 | ANALYSIS QUESTIONS**Q1. Why does BFS find a shortest path in an unweighted graph?**

BFS expands nodes in order of depth (number of edges from the start), because its FIFO queue keeps all depth-d nodes ahead of any depth-(d+1) node. The first time the goal is reached, no shallower route exists. In an unweighted graph path cost = number of edges, so this path is also the cheapest.

Q2. Why does DFS not guarantee the shortest path?

DFS commits to the most recent branch and follows it as deep as possible before trying alternatives. It returns the first goal it happens to reach on that branch, which depends on the action order, not on depth or cost. In our maze DFS returned 9 edges / cost 11 although a 3-edge path and a cost-9 path exist.

Q3. Why does UCS consider path cost?

In weighted problems actions have different costs (distance, time, energy), so the path with fewest steps may be expensive. UCS orders its frontier by $g(n)$, always expanding the cheapest unexpanded node. When the goal is selected for expansion, no cheaper path to it can exist, so the solution is least-cost.

Q4. Construct an example where BFS returns a higher-cost path than UCS.

Graph: S–G cost 5, S–A cost 1, A–G cost 1.

BFS: S → G (1 edge, cost 5). UCS: S → A → G (2 edges, cost 2).

Our maze is a larger example: BFS returns S→D→F→G (cost 12), UCS returns the 9-edge route of cost 9.

Q5. What happens when all edge costs are equal?

If every step cost is the same constant c , then $g(n) = c \times \text{depth}(n)$. UCS then expands nodes in order of depth, i.e. it behaves like BFS (with extra priority-queue overhead). Both return an optimal path, possibly a different one of equal length depending on tie-breaking.

Q6. Why is a visited/explored mechanism important?

Without it the same state can be generated and expanded many times through different paths or cycles (e.g. S→A→S→A ...). This wastes time and memory, can make the search exponential, and in the case of DFS can make it run forever. The explored set ensures each state is expanded at most once.

Q7. What can happen if DFS is implemented without cycle detection?

DFS can get trapped in an infinite loop. In our maze, with action order Up, Left, Right, Down, DFS would go S→A→S→A ... forever (moving Left from A returns to S), never reaching G. Even without an infinite loop it would re-expand states repeatedly.

Q8. Compare BFS and DFS in terms of memory usage.

BFS stores the whole frontier level: $O(b^d)$ nodes, where b = branching factor and d = solution depth — memory grows exponentially.

DFS stores only the current path plus the unexpanded siblings along it: $O(b \cdot m)$, where m = maximum depth — linear memory. (With a full explored set, graph-search DFS can still store all visited states.)

Q9. Under what conditions is UCS optimal?

UCS is optimal (and complete) when every step cost is at least some small positive constant $\epsilon > 0$, the branching factor is finite, and the goal test is applied when a node is selected for expansion (not when it is generated). Negative costs break optimality; zero-cost cycles can break completeness.

Q10. Which algorithm would you choose for a problem where edge costs differ significantly? Explain.

Uniform-Cost Search (or A* if a good admissible heuristic is available). BFS and DFS ignore costs and can return very expensive solutions, as BFS did here (12 vs 9). UCS guarantees the least-cost solution; A* gives the same guarantee while usually expanding fewer nodes.

SECTION 12 | STUDENT IMPLEMENTATION CHECKLIST

☒ Problem formulated	☒ UCS implemented
☒ Initial state defined	☒ Search trace generated
☒ Goal state defined	☒ Final paths verified
☒ State-space created	☒ Costs calculated
☒ Edge costs assigned	☒ Algorithms compared
☒ BFS implemented	☒ Questions answered
☒ DFS implemented	

SECTION 13 | SUBMISSION REQUIREMENTS

#	Deliverable	In this worked example	✓
1	Python source code	<code>maze_search.py</code> (Appendix A) and <code>maze_extension.py</code> (Appendix B)	☒
2	Completed work file	This document	☒
3	State-space / graph diagram	Section 5, Figure 1	☒
4	BFS trace	Section 9.1	☒
5	DFS trace	Section 9.2	☒
6	UCS trace	Section 9.3	☒
7	Comparison table	Section 10	☒
8	Answers to analysis questions	Section 11	☒
9	Screenshots of program execution	Below	☒

Screenshots of Program Execution

Screenshot 1 — Maze and adjacency list

```
$ python maze_search.py    (maze & graph)

Maze (terrain)              Maze (state names)
S . . .                    S A B C
W # # .                    D # # E
. # M .                    F # H I
G . . .                    G J K L
Costs: '.'=1 M(mud)=3 W(water)=10 #=wall

Adjacency list (neighbour: step cost, action)
S -> A:1 Right, D:10 Down
A -> S:1 Left, B:1 Right
B -> A:1 Left, C:1 Right
C -> B:1 Left, E:1 Down
D -> S:1 Up, F:1 Down
E -> C:1 Up, I:1 Down
F -> D:10 Up, G:1 Down
H -> I:1 Right, K:1 Down
I -> E:1 Up, H:3 Left, L:1 Down
G -> F:1 Up, J:1 Right
J -> G:1 Left, K:1 Right
K -> H:3 Up, J:1 Left, L:1 Right
L -> I:1 Up, K:1 Left
```

Screenshot 2 — BFS output and comparison

```
$ python maze_search.py    (BFS output)

--- BFS trace ---
Step Node  Frontier      Explored      Current path      Cost
1   S     A, D         S             S                 0
2   A     D, B         S,A           S-A               1
3   D     B, F         S,A,D         S-D               10
4   B     F, C         S,A,D,B       S-A-B             2
5   F     C, G         S,A,D,B,F     S-D-F             11
6   C     G, E         S,A,D,B,F,C   S-A-B-C           3
7   G     E           S,A,D,B,F,C,G S-D-F-G           12

=== Comparison ===
Algorithm Returned path      Length  Cost  Expanded  Optimal?
BFS      S -> D -> F -> G      3      12    7         No
DFS      S -> A -> B -> C -> E -> I -> H -> K -> J -> G      9      11    10        No
UCS      S -> A -> B -> C -> E -> I -> L -> K -> J -> G      9      9     11        Yes
```

Screenshot 3 — DFS output and comparison

```
$ python maze_search.py    (DFS output)

--- DFS trace ---
Step Node   Frontier           Explored           Current path       Cost
1   S      D, A             S                 S                 0
2   A      D, B             S,A              S-A               1
3   B      D, C             S,A,B            S-A-B             2
4   C      D, E             S,A,B,C          S-A-B-C           3
5   E      D, I             S,A,B,C,E        S-A-B-C-E         4
6   I      D, L, H          S,A,B,C,E,I      S-A-B-C-E-I       5
7   H      D, L, K          S,A,B,C,E,I,H    S-A-B-C-E-I-H     8
8   K      D, L, L, J       S,A,B,C,E,I,H,K  S-A-B-C-E-I-H-K   9
9   J      D, L, L, G       S,A,B,C,E,I,H,K,J S-A-B-C-E-I-H-K-J 10
10  G      D, L, L          S,A,B,C,E,I,H,K,J,G S-A-B-C-E-I-H-K-J-G 11

=== Comparison ===
Algorithm Returned path                               Length Cost Expanded Optimal?
BFS       S -> D -> F -> G                               3    12      7        No
DFS       S -> A -> B -> C -> E -> I -> H -> K -> J -> G       9     11     10        No
UCS       S -> A -> B -> C -> E -> I -> L -> K -> J -> G       9     9      11        Yes
```

Screenshot 4 — UCS output and comparison

```
$ python maze_search.py    (UCS output)

--- UCS trace ---
Step Node   Frontier           Explored           Current path       Cost
1   S      A(1), D(10)        S                 S                 0
2   A      B(2), D(10)        S,A              S-A               1
3   B      C(3), D(10)        S,A,B            S-A-B             2
4   C      E(4), D(10)        S,A,B,C          S-A-B-C           3
5   E      I(5), D(10)        S,A,B,C,E        S-A-B-C-E         4
6   I      L(6), H(8), D(10)  S,A,B,C,E,I      S-A-B-C-E-I       5
7   L      K(7), H(8), D(10)  S,A,B,C,E,I,L    S-A-B-C-E-I-L     6
8   K      H(8), J(8), D(10)  S,A,B,C,E,I,L,K  S-A-B-C-E-I-L-K   7
9   H      J(8), D(10)        S,A,B,C,E,I,L,K,H S-A-B-C-E-I-H     8
10  J      G(9), D(10)        S,A,B,C,E,I,L,K,H,J S-A-B-C-E-I-L-K-J 8
11  G      D(10)              S,A,B,C,E,I,L,K,H,J,G S-A-B-C-E-I-L-K-J-G 9

=== Comparison ===
Algorithm Returned path                               Length Cost Expanded Optimal?
BFS       S -> D -> F -> G                               3    12      7        No
DFS       S -> A -> B -> C -> E -> I -> H -> K -> J -> G       9     11     10        No
UCS       S -> A -> B -> C -> E -> I -> L -> K -> J -> G       9     9      11        Yes
```

SECTION 14 | ASSESSMENT RUBRIC

Total: 20 Marks (left blank — used by the instructor to grade student submissions)

Criterion	Marks	Excellent	Satisfactory	Needs Improvement	Score
Problem formulation	4	All components precise and consistent; graph meets every requirement.	Most components correct; minor gaps.	Components missing, vague or inconsistent.	
BFS implementation	3	Own code; correct FIFO frontier, explored set, path reconstruction.	Works with minor issues.	Incorrect, incomplete or library-based.	
DFS implementation	3	Own code; correct LIFO behaviour and cycle handling.	Works with minor issues.	Loops, incorrect or library-based.	
UCS implementation	4	Correct priority queue, cumulative cost, best-cost tracking; optimal result.	Mostly correct; minor cost-handling issues.	Not cost-ordered or incorrect.	
Tracing and comparison	3	Complete traces from real execution; insightful comparison.	Traces present but partial.	Missing or inconsistent with code.	
Analysis / report	3	Clear, correct, well-reasoned answers with examples.	Mostly correct; limited depth.	Incomplete or incorrect answers.	
TOTAL	20				/ 20

SECTION 15 | EXTENSION CHALLENGE (OPTIONAL)

Challenge A — Expansion-Order Visualization

Task

“Modify your search program so that it **records and visualizes the order in which nodes are expanded.**”

Approach and visualization

Each algorithm already returns its trace; the first column of the trace is the expansion order.

`maze_extension.py` (Appendix B) draws the maze with Matplotlib, writes the expansion number in each expanded cell and overlays the returned path in red.

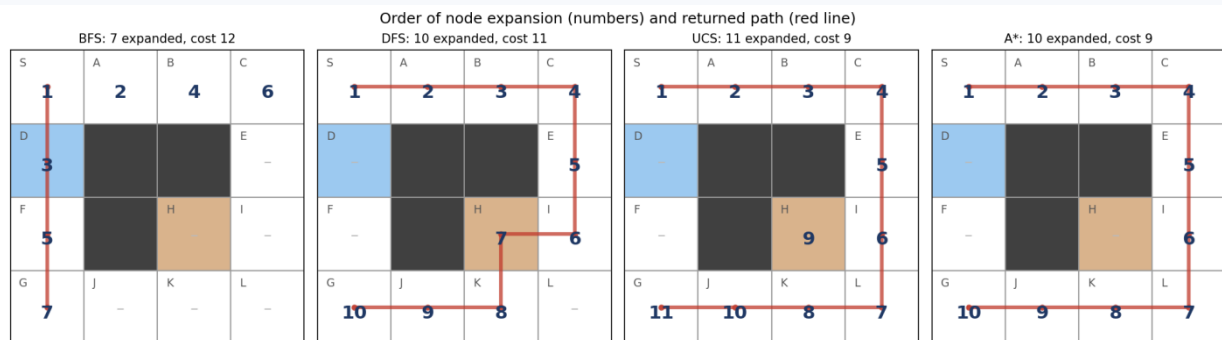


Figure 2. Expansion order (numbers) and returned path (red) for BFS, DFS, UCS and A*.

Challenge B — A* Search**Task**

“Modify the problem to support A* Search by adding a heuristic function.”

Heuristic $h(n)$ definition	Manhattan distance to the goal: $h(n) = \text{row}(n) - \text{row}(G) + \text{col}(n) - \text{col}(G)$. Values: S=3, A=4, B=5, C=6, D=2, E=5, F=1, H=3, I=4, G=0, J=1, K=2, L=3
Is $h(n)$ admissible? Why?	Yes. Each move changes the row or column by exactly 1 and costs at least 1, so at least $h(n)$ moves and at least $h(n)$ cost are needed to reach G. h never overestimates (it is also consistent).
A* vs UCS: nodes expanded & path cost	A*: S → A → B → C → E → I → L → K → J → G, cost 9, 10 nodes expanded. UCS: same path, cost 9, 11 nodes expanded. A* skipped the mud cell H, because $f(H) = 8 + 3 = 11$ is higher than $f(G) = 9$. The saving is small here because the maze is tiny and the heuristic is misleading at first (it points towards the water); on larger grids the difference is much bigger.

APPENDIX A | COMPLETE PROGRAM — maze_search.py

Complete, runnable sample program for the maze problem. It uses only the Python standard library. Run it with `python maze_search.py`; the output is shown in the screenshots in Section 13.

```

1  """
2  =====
3  Artificial Intelligence - Implementation Task 1 (Sample Solution)
4  Maze / Grid Search Problem solved with BFS, DFS and UCS
5  =====
6  University of Okara - Department of Computer Science
7  Instructor: Dr. Ghulam Ali, Associate Professor of Computer Science
8  =====
9
10 PROBLEM
11     An agent stands at START (S) in a 4 x 4 maze and must reach GOAL (G).
12     It can move one cell Up, Left, Right or Down. It cannot enter walls (#).
13     Entering a cell costs according to its terrain:
14     '.' normal floor = 1, 'M' mud = 3, 'W' water = 10
15     The shortest route (fewest moves) goes through the water cell,
16     so the path with fewer edges is NOT the least-cost path.
17
18 Run:  python maze_search.py
19 Only the Python standard library is used (collections.deque, heapq).
20 """
21
22 from collections import deque
23 import heapq
24
25 # -----
26 # 1. MAZE DEFINITION
27 # -----
28 MAZE = [
29     "S...",
30     "W##.",
31     ".#M.",
32     "G...",
33 ]
34
35 TERRAIN_COST = {"." : 1, "S" : 1, "G" : 1, "M" : 3, "W" : 10}
36
37 # Actions in the order they are generated (this order matters for DFS).
38 ACTIONS = [
39     ("Up",    (-1, 0)),
40     ("Left",  (0, -1)),
41     ("Right", (0, 1)),
42     ("Down",  (1, 0)),
43 ]
44
45 # Letters used to name open cells (G is reserved for the Goal).
46 CELL_LETTERS = "ABCDEFGHIJKLNOPQRSTUVWXYZ"
47
48 # -----
49 # 2. PROBLEM FORMULATION -> STATE-SPACE GRAPH
50 # -----
51 # -----
52 def label_cells(maze):
53     """Give every open cell a readable name: S, G, or a letter A, B, C ...
54     Returns two dicts: (row, col) -> name and name -> (row, col)."""
55     letters = iter(CELL_LETTERS)
56     pos_to_name, name_to_pos = {}, {}
57     for r, row in enumerate(maze):
58         for c, ch in enumerate(row):
59             if ch == "#":
60                 continue # walls are not states
61             name = ch if ch in "SG" else next(letters)
62             pos_to_name[(r, c)] = name
63             name_to_pos[name] = (r, c)
64     return pos_to_name, name_to_pos
65
66
67 def build_graph(maze):
68     """Transition model: from each open cell, apply every action that stays
69     inside the maze and does not hit a wall. The step cost is the terrain
70     cost of the cell being ENTERED.
71     Result: graph[name] = [(neighbour_name, step_cost, action), ...]"""
72     pos_to_name, name_to_pos = label_cells(maze)
73     rows, cols = len(maze), len(maze[0])
74     graph = {}
75     for (r, c), name in pos_to_name.items():
76         graph[name] = []
77         for action, (dr, dc) in ACTIONS:
78             nr, nc = r + dr, c + dc
79             if 0 <= nr < rows and 0 <= nc < cols and maze[nr][nc] != "#":
80                 cost = TERRAIN_COST[maze[nr][nc]]
81                 graph[name].append((pos_to_name[(nr, nc)], cost, action))

```

```

82     return graph, name_to_pos
83
84
85 def path_cost(graph, path):
86     """Sum of step costs along a path."""
87     total = 0
88     for a, b in zip(path, path[1:]):
89         total += next(cost for (n, cost, _) in graph[a] if n == b)
90     return total
91
92
93 def reconstruct(parent, goal):
94     """Follow parent pointers back from the goal to the start."""
95     path = [goal]
96     while parent[path[-1]] is not None:
97         path.append(parent[path[-1]])
98     return path[::-1]
99
100
101 # -----
102 # 3. BREADTH-FIRST SEARCH (FIFO queue)
103 # -----
104 def bfs(graph, start, goal):
105     frontier = deque([start])          # FIFO queue
106     parent = {start: None}             # also acts as the "reached" set
107     explored = []                      # expansion order
108     trace = []
109
110     while frontier:
111         node = frontier.popleft()       # oldest node first
112         explored.append(node)
113         current_path = reconstruct(parent, node)
114
115         if node == goal:                # goal test when the node is expanded
116             trace.append((node, list(frontier), list(explored),
117                           current_path, path_cost(graph, current_path)))
118             return current_path, len(explored), trace
119
120         for child, _, _ in graph[node]:
121             if child not in parent:     # not explored and not in frontier
122                 parent[child] = node
123                 frontier.append(child)
124
125         trace.append((node, list(frontier), list(explored),
126                       current_path, path_cost(graph, current_path)))
127     return None, len(explored), trace
128
129
130 # -----
131 # 4. DEPTH-FIRST SEARCH (LIFO stack, graph search)
132 # -----
133 def dfs(graph, start, goal):
134     frontier = [(start, [start])]      # stack of (node, path to node)
135     explored = []
136     explored_set = set()               # cycle detection
137     trace = []
138
139     while frontier:
140         node, path = frontier.pop()    # newest node first
141         if node in explored_set:       # already expanded via another path
142             continue
143         explored.append(node)
144         explored_set.add(node)
145
146         if node == goal:
147             trace.append((node, [n for n, _ in frontier], list(explored),
148                           path, path_cost(graph, path)))
149             return path, len(explored), trace
150
151         # Push children in REVERSE action order so the first action
152         # (Up, then Left, Right, Down) ends up on top of the stack.
153         for child, _, _ in reversed(graph[node]):
154             if child not in explored_set:
155                 frontier.append((child, path + [child]))
156
157         trace.append((node, [n for n, _ in frontier], list(explored),
158                       path, path_cost(graph, path)))
159     return None, len(explored), trace
160
161
162 # -----
163 # 5. UNIFORM-COST SEARCH (priority queue ordered by g(n))
164 # -----
165 def ucs(graph, start, goal):
166     counter = 0                        # tie-breaker: FIFO among equal costs
167     frontier = [(0, counter, start)]   # (g, order, node)
168     best_cost = {start: 0}             # cheapest known cost to each node
169     parent = {start: None}
170     explored = []

```

```

171     explored_set = set()
172     trace = []
173
174     def frontier_view():
175         """Live frontier entries (stale duplicates hidden), cheapest first."""
176         live = [(g, k, n) for (g, k, n) in frontier
177                 if n not in explored_set and g == best_cost[n]]
178         return [f"{n}({g})" for g, k, n in sorted(live)]
179
180     while frontier:
181         g, _, node = heapq.heappop(frontier) # lowest path cost first
182         if node in explored_set or g > best_cost[node]:
183             continue # stale entry - skip
184         explored.append(node)
185         explored_set.add(node)
186         current_path = reconstruct(parent, node)
187
188         if node == goal:
189             trace.append((node, frontier_view(), list(explored), current_path, g))
190             return current_path, g, len(explored), trace
191
192         for child, step, _ in graph[node]:
193             new_cost = g + step
194             if child not in explored_set and new_cost < best_cost.get(child, float("inf")):
195                 best_cost[child] = new_cost # found a cheaper way
196                 parent[child] = node
197                 counter += 1
198                 heapq.heappush(frontier, (new_cost, counter, child))
199
200         trace.append((node, frontier_view(), list(explored), current_path, g))
201     return None, float("inf"), len(explored), trace
202
203
204 # -----
205 # 6. OUTPUT HELPERS
206 # -----
207 def show_maze(maze, name_to_pos):
208     print("Maze (terrain)          Maze (state names)")
209     names = {pos: n for n, pos in name_to_pos.items()}
210     for r, row in enumerate(maze):
211         left = " ".join(row)
212         right = " ".join(names.get((r, c), "#") for c in range(len(row)))
213         print(f"    {left:<22}    {right}")
214     print("Costs: '.'=1 M(mud)=3 W(water)=10 #=wall\n")
215
216
217 def show_graph(graph):
218     print("Adjacency list (neighbour: step cost, action)")
219     for node, edges in graph.items():
220         text = ", ".join(f"{n}:{c} {a}" for n, c, a in edges)
221         print(f"    {node} -> {text}")
222     print()
223
224
225 def show_trace(title, trace, frontier_is_text=False):
226     print(f"--- {title} trace ---")
227     print(f"{'Step':<5}{Node':<6}{Frontier':<26}{Explored':<26}{Current path':<22}{Cost':>4}")
228     for i, (node, front, expl, path, cost) in enumerate(trace, 1):
229         f = ", ".join(front) if front else "-"
230         print(f"{i:<5}{node:<6}{f:<26}{', '.join(expl):<26}{'-'.join(path):<22}{cost:>4}")
231     print()
232
233
234 # -----
235 # 7. MAIN PROGRAM
236 # -----
237 def main():
238     graph, name_to_pos = build_graph(MAZE)
239     start, goal = "S", "G"
240
241     show_maze(MAZE, name_to_pos)
242     show_graph(graph)
243
244     bfs_path, bfs_n, bfs_trace = bfs(graph, start, goal)
245     dfs_path, dfs_n, dfs_trace = dfs(graph, start, goal)
246     ucs_path, ucs_cost, ucs_n, ucs_trace = ucs(graph, start, goal)
247
248     show_trace("BFS", bfs_trace)
249     show_trace("DFS", dfs_trace)
250     show_trace("UCS", ucs_trace)
251
252     results = [
253         ("BFS", bfs_path, bfs_n),
254         ("DFS", dfs_path, dfs_n),
255         ("UCS", ucs_path, ucs_n),
256     ]
257     optimal = path_cost(graph, ucs_path) # UCS is optimal (costs > 0)
258
259     print("=== Comparison ===")

```

```
260     print(f'{Algorithm':<10}{'Returned path':<42}{'Length':>7}{'Cost':>6}{'Expanded':>10}{'Optimal?':>10}")
261     for name, path, n in results:
262         cost = path_cost(graph, path)
263         print(f'{name':<10}{' -> '.join(path):<42}{'len(path) - 1':>7}{'cost':>6}{'n':>10}"
264               f'{'Yes' if cost == optimal else 'No':>10}")
265
266
267 if __name__ == "__main__":
268     main()
```

APPENDIX B | EXTENSION PROGRAM — maze_extension.py

Expansion-order visualization and A* Search (Section 15). Needs `matplotlib` and `maze_search.py` in the same folder. Run with `python maze_extension.py`; it saves `expansion_order.png`.

```

1  """
2  Extension Challenge (Sample Solution) - Implementation Task 1
3  (a) Records and visualizes the ORDER in which nodes are expanded
4      by BFS, DFS and UCS (Matplotlib).
5  (b) Adds A* Search with the Manhattan-distance heuristic.
6
7  Requires: maze_search.py in the same folder, and matplotlib.
8  Run:      python maze_extension.py
9  """
10 import heapq
11 import matplotlib.pyplot as plt
12 from matplotlib.patches import Rectangle
13 from maze_search import MAZE, build_graph, bfs, dfs, ucs, path_cost, reconstruct
14
15 FILL = {".": "#FFFFFF", "S": "#FFFFFF", "G": "#FFFFFF", "M": "#D9B38C", "W": "#9CC9F0", "#": "#404040"}
16
17
18 # -----
19 # (b) A* SEARCH  f(n) = g(n) + h(n)
20 # -----
21 def manhattan(name_to_pos, goal):
22     """h(n) = |row - goal_row| + |col - goal_col|.
23     Admissible because every move costs at least 1 and at least
24     this many moves are needed to reach the goal."""
25     gr, gc = name_to_pos[goal]
26     return lambda n: abs(name_to_pos[n][0] - gr) + abs(name_to_pos[n][1] - gc)
27
28
29 def astar(graph, start, goal, h):
30     counter = 0
31     frontier = [(h(start), counter, 0, start)]    # (f, order, g, node)
32     best_g = {start: 0}
33     parent = {start: None}
34     explored = []
35     while frontier:
36         f, _, g, node = heapq.heappop(frontier)
37         if node in explored or g > best_g[node]:
38             continue
39         explored.append(node)
40         if node == goal:
41             return reconstruct(parent, node), g, explored
42         for child, step, _ in graph[node]:
43             ng = g + step
44             if child not in explored and ng < best_g.get(child, float("inf")):
45                 best_g[child] = ng
46                 parent[child] = node
47                 counter += 1
48                 heapq.heappush(frontier, (ng + h(child), counter, ng, child))
49     return None, float("inf"), explored
50
51
52 # -----
53 # (a) EXPANSION-ORDER VISUALIZATION
54 # -----
55 def draw_panel(ax, name_to_pos, order, path, title):
56     rows, cols = len(MAZE), len(MAZE[0])
57     names = {p: n for n, p in name_to_pos.items()}
58     rank = {n: i + 1 for i, n in enumerate(order)}
59     for r in range(rows):
60         for c in range(cols):
61             ch = MAZE[r][c]
62             ax.add_patch(Rectangle((c, rows - 1 - r), 1, 1, facecolor=FILL[ch], edgecolor="#999999"))
63             n = names.get((r, c))
64             if n is None:
65                 continue
66             ax.text(c + 0.12, rows - 1 - r + 0.78, n, fontsize=9, color="#555555")
67             if n in rank:
68                 # expansion number
69                 ax.text(c + 0.5, rows - 1 - r + 0.42, str(rank[n]), fontsize=15,
70                        ha="center", va="center", weight="bold", color="#1F3864")
71             else:
72                 ax.text(c + 0.5, rows - 1 - r + 0.42, "-", fontsize=13, ha="center", color="#BBBBBB")
73     xs = [name_to_pos[n][1] + 0.5 for n in path]
74     ys = [rows - 1 - name_to_pos[n][0] + 0.5 for n in path]
75     ax.plot(xs, ys, color="#C0392B", lw=3, alpha=0.75, marker="o", ms=4)
76     ax.set_xlim(0, cols); ax.set_ylim(0, rows); ax.set_aspect("equal")
77     ax.set_xticks([]); ax.set_yticks([])
78     ax.set_title(title, fontsize=10)
79
80
81 def main():
82     graph, name_to_pos = build_graph(MAZE)

```

```

82 s, g = "S", "G"
83 bp, bn, bt = bfs(graph, s, g)
84 dp, dn, dt = dfs(graph, s, g)
85 up, uc, un, ut = ucs(graph, s, g)
86 h = manhattan(name_to_pos, g)
87 ap, ac, aorder = astar(graph, s, g, h)
88
89 runs = [("BFS", [t[0] for t in bt], bp), ("DFS", [t[0] for t in dt], dp),
90         ("UCS", [t[0] for t in ut], up), ("A*", aorder, ap)]
91 print("Heuristic values h(n):", {n: h(n) for n in graph})
92 for name, order, path in runs:
93     print(f"{name:<4} expansion order: {' '.join(order):<24} path: {'-'.join(path)}"
94           f" cost={path_cost(graph, path)} expanded={len(order)}")
95
96 fig, axes = plt.subplots(1, 4, figsize=(14, 4))
97 for ax, (name, order, path) in zip(axes, runs):
98     draw_panel(ax, name_to_pos, order, path,
99               f"{name}: {len(order)} expanded, cost {path_cost(graph, path)}")
100 fig.suptitle("Order of node expansion (numbers) and returned path (red line)", fontsize=12)
101 fig.tight_layout()
102 fig.savefig("expansion_order.png", dpi=150)
103 plt.show()
104
105
106 if __name__ == "__main__":
107     main()

```

— End of Implementation Task 1 (Worked Example) —